https://chatgpt.com/share/6af6a0df-0758-83ec-8538-6d276171ba80

Ask: explain how inheritance works between staff, waiter, chef & cashier

ChatGPT:

In your SC2002 Restaurant Rush project, **inheritance allows `Waiter`, `Chef`, and `Cashier` to share common staff behaviour without repeating the same code.**

The main idea is that all three are staff members, but each has a different responsibility.

1. How the four classes are connected

The hollow-arrow relationship points towards the parent class, Staff.

- `Staff` is the parent class (superclass).
- `Waiter`, `Chef`, `Cashier` are child classes (subclasses).
- Each child class inherits the common functionality from `Staff`, while having its own specialised methods.

Importantly, `Waiter`, `Chef` and `Cashier` do **not** inherit from one another. They all inherit directly from `Staff`.

2. What does Staff.java do?

Your `Staff` class is an abstract superclass that centralises the common task-limit behaviour.

In particular, these are the shared methods:

Method	Purpose
`canAct()`	Checks whether the staff member can still perform a task this turn.
`markTaskUsed()`	Marks the staff member's task as used.

`resetForTurn()`	Resets the task allowance at the beginning of a new turn.

For example, a simplified illustration of the shared logic is:

```
None
public abstract class Staff {
    private boolean taskUsed = false;

    public boolean canAct() {
        return !taskUsed;
    }

    protected void markTaskUsed() {
        taskUsed = true;
    }

    public void resetForTurn() {
        taskUsed = false;
    }
}
```

This illustrates the concept rather than reproducing your complete `Staff.java` verbatim.

Why is this useful?

Without inheritance, you would need to implement these same methods separately in `Waiter`, `Chef` and `Cashier`.

With inheritance, you write them **once inside** `Staff`, and all three subclasses can reuse them.

3. How each subclass uses inheritance

Waiter.java

```
None
public class Waiter extends Staff
```

Inherits the task management methods, but implements its own actions: `seatCustomer()`, `takeOrder()` and `serveOrder()`.

For example, once the Waiter seats a customer, `markTaskUsed()` prevents it from taking another task during that turn.

Chef.java

```
None
public class Chef extends Staff
```

Inherits the same task management methods but has its own `prepareOrder()` method.

The Chef can prepare one unit of food per turn. Once preparation is performed, it uses `markTaskUsed()` to prevent another action that turn.

Cashier.java

```
None
public class Cashier extends Staff
```

Inherits the same task management methods but implements `handlePayment()`.

Unlike the Waiter and Chef, the Cashier automatically processes eligible payments, handling at most one customer per turn.

4. Example: How inheritance works during one game turn

Imagine your restaurant has a customer ordering a Burger.

1. **Start of turn**
   `resetForTurn()` is called for the Waiter, Chef and Cashier. All three can now perform one task.
2. **Waiter takes an order**
   The Waiter calls its specialised `takeOrder()` method and then the inherited

`markTaskUsed()` method.
`waiter.canAct()` now returns `false`.

3. **Chef prepares the Burger**
   The Chef calls `prepareOrder()` and then `markTaskUsed()`.
   `chef.canAct()` now returns `false`.
4. **Cashier handles an eligible payment**
   If another customer is already ready to pay, the Cashier processes that payment and uses its task allowance.

Each object has its own task state. Using the Waiter's task does not consume the Chef's or Cashier's task.

Notice something important: **the methods are shared, but the objects are independent.**

Although all three staff types inherit `canAct()`, the Waiter using its task does not prevent the Chef from preparing food.

Each object has its own inherited task state.

5. How this demonstrates OOP principles

Your design primarily demonstrates these three concepts:

Concept	How your code demonstrates it
Inheritance	`Waiter`, `Chef` and `Cashier` extend `Staff` to reuse common functionality.
Encapsulation	The task state is managed internally by `Staff` rather than directly modified by other classes.
Abstraction	The abstract `Staff` class represents common staff behaviour, while subclasses handle their specific responsibilities.

One important distinction: inheritance alone does not necessarily demonstrate **polymorphism through method overriding**. That would require subclasses to provide different

implementations of a shared method, or otherwise be used through a common parent type. Simply inheriting `canAct()` is code reuse.

6. How to explain this in your project presentation

> For Stage 1, I implemented a Staff superclass and three subclasses: Waiter, Chef and Cashier. I used inheritance because all three staff types share the same task limit of one action per turn. Instead of repeating the logic in each class, I placed common methods like `canAct()`, `markTaskUsed()` and `resetForTurn()` inside Staff. Each subclass then implements its own responsibilities, such as the Waiter handling orders, the Chef preparing food and the Cashier processing payments. This reduces duplicate code and makes the design easier to maintain and extend.

The most important thing to understand: `Staff` determines *whether a staff member can perform another task*, while `Waiter`, `Chef` and `Cashier` determine *what that task actually does*.

That separation of common behaviour and specialised responsibilities is the main reason inheritance makes sense in your project.

Ask: should staff be an abstract class? explain why

ChatGPT:

Yes, I would recommend making `Staff` an abstract class in your SC2002 Restaurant Rush project. It makes sense from an OOP design perspective because `Staff` represents a general category of employees rather than a specific role.

1. Why should Staff be abstract?

Think about your restaurant in real life.

You can hire a Waiter, Chef or Cashier, but you wouldn't hire someone whose job is simply "Staff" without assigning them a role.

The same idea applies in Java.

- `Staff` represents the general concept of an employee.
- `Waiter`, `Chef` and `Cashier` represent specific employee roles.
- Every employee shares certain behaviours, such as being limited to one task per turn.

Therefore, `Staff` is useful as a parent class, but **there is no need to create a generic `Staff` object directly**.

2. Abstract vs non-abstract Staff

Consider these two implementations.

Option A: Normal class

Less suitable

```
None
public class Staff {
    // Shared staff methods
}
```

Java allows:

```
None
Staff staff = new Staff();
```

The problem is that `staff` has no specific restaurant role. What job is it supposed to perform?

Option B: Abstract class

Recommended

```
None
public abstract class Staff {
    // Shared staff methods
}
```

Java prevents:

```
None
Staff staff = new Staff(); // ERROR
```

But allows:

```
None
Staff waiter = new Waiter();
Staff chef = new Chef();
Staff cashier = new Cashier();
```

This ensures that the actual objects are concrete staff roles.

3. Does Staff need abstract methods?

No. This is an important distinction.

An abstract class does not necessarily need abstract methods.

For example:

```
None
public abstract class Staff {
    private boolean taskUsed = false;

    public boolean canAct() {
        return !taskUsed;
    }

    protected void markTaskUsed() {
        taskUsed = true;
    }

    public void resetForTurn() {
        taskUsed = false;
    }
}
```

This is perfectly valid Java.

All three methods have implementations that `Waiter`, `Chef` and `Cashier` can inherit directly.

You only need abstract methods if you want to force subclasses to implement certain behaviours.

For example:

```
None
public abstract void performTask();
```

This would require each concrete subclass to implement `performTask()`.

However, **I would not add this method unnecessarily** because your Waiter, Chef and Cashier have different task types and method parameters. The assignment specifically advises against introducing abstractions just to satisfy an OOP checklist.

4. Why is abstract Staff suitable for your project?

Reason	Explanation
Prevents meaningless objects	You cannot instantiate a generic `Staff` without a specific role.
Promotes code reuse	Shared task-management logic is implemented once.
Supports extensibility	Stage 3 can introduce `Host extends Staff` without duplicating the common logic.
Improves design clarity	Separates general staff behaviour from specialised responsibilities.

The Stage 3 extension is especially useful evidence for your design: introducing a Host is easier when the common staff task-management logic already exists.

One qualification: the game could still function correctly with a normal `Staff` class. Making it abstract is a **design choice**, not a Java requirement or an explicit assignment requirement.

5. Exam/project presentation-ready explanation

> I made `Staff` an abstract class because it represents a general employee rather than a specific role in the restaurant. We should only instantiate concrete roles such as Waiter, Chef and Cashier. The abstract class also allows these subclasses to inherit

common methods for managing their task limits, reducing duplicated code. This makes the design easier to maintain and extend, especially when introducing the Host in Stage 3.

My recommendation: Keep `Staff` abstract, but don't introduce abstract methods unless your design genuinely needs them. This is a simple, justified use of abstraction and inheritance that you can defend in your SC2002 report.

Ask:
heres my staff.java. check whether my implementation follows encapsulation

```
public abstract class Staff {

    private int tasksUsed = 0;
    private final int maxTask = 1;

    public void resetForTurn() {
        tasksUsed = 0;
    }

    public boolean canAct() {
        return tasksUsed < maxTask;
    }

    protected void markTaskUsed() {
        if (!canAct()) {
            throw new IllegalStateException(
                "Staff has already completed their task this turn."
            );
        }

        tasksUsed++;
    }
}
```

ChatGPT:

Yes, your `Staff.java` demonstrates encapsulation correctly. In fact, I would say it's a good example of encapsulation for your SC2002 project.

Let's go through exactly why.

1. Encapsulation through private attributes

```
None
private int tasksUsed = 0;
private final int maxTask = 1;
```

Both attributes are declared `private`, meaning they cannot be accessed or modified directly by other classes, including `Waiter`, `Chef` and `Cashier`.

For example, this would produce a compilation error:

```
None
Waiter waiter = new Waiter();
waiter.tasksUsed = 5; // ERROR
```

Instead, the attributes can only be managed through methods defined within `Staff`.

This protects your object's internal state from unintended modification.

2. Controlled access through methods

Your class provides three methods that control how the internal state is accessed or changed.

Method	Access modifier	Purpose
`resetForTurn()`	`public`	Resets `tasksUsed` to 0.
`canAct()`	`public`	Checks whether the staff member has remaining tasks.
`markTaskUsed()`	`protected`	Increments `tasksUsed`, but only when a task is available.

This is encapsulation because other classes interact with the staff object's state **through methods rather than accessing its attributes directly**.

Why `markTaskUsed()` is especially good

```
None
protected void markTaskUsed() {
    if (!canAct()) {
        throw new IllegalStateException(
            "Staff has already completed their task this
turn."
        );
    }

    tasksUsed++;
}
```

This method protects an important rule in your restaurant game: staff members cannot exceed their task limit.

For example:

```
None
markTaskUsed(); // tasksUsed becomes 1
markTaskUsed(); // Throws IllegalStateException
```

Without this check, you could accidentally allow a Waiter to perform multiple tasks in one turn.

This is stronger encapsulation than simply making your attributes private, because your methods actively enforce the object's rules.

3. Why is `markTaskUsed()` protected instead of public?

This is a sensible design choice because `Staff` is an abstract superclass.

In Java:

- `private` means only the declaring class can access it.

- `protected` allows subclasses such as `Waiter`, `Chef` and `Cashier` to access the method, as well as classes in the same package.
- `public` allows any class to access the method.

By using `protected`, your subclasses can call `markTaskUsed()` when completing their tasks, without exposing that method as part of the fully public API.

One small caveat: Java's `protected` also permits access from other classes within the same package, so it is not exclusively limited to subclasses.

4. Is there anything you should improve?

Only one minor improvement I would suggest:

```
None
private static final int MAX_TASKS = 1;
```

Instead of:

```
None
private final int maxTask = 1;
```

Both are valid, but `static final` is more appropriate if the limit of one task is identical for every staff member.

It also follows Java's naming convention for constants.

However, if you intend to allow different staff roles to have different task limits in the future, keeping an instance-specific `final` field may make more sense.

One other point: because `resetForTurn()` is public, any class can reset a staff member's task count. This is acceptable if your game controller is responsible for calling it once per turn, but the controller must use it correctly. Encapsulation does not automatically prevent every possible misuse of a public method.

5. How I would assess your implementation

OOP aspect	Assessment
Private attributes	Correct
Controlled access through methods	Correct
Validation before modifying state	Correct
Appropriate access modifiers	Good
Protection of class invariants	Good, with the reset-method caveat
Overall encapsulation	**9/10**

Project report-ready explanation

> I applied encapsulation in the `Staff` class by declaring `tasksUsed` and `maxTask` as private attributes. This prevents other classes from directly modifying the staff's task count. Instead, I provided methods such as `canAct()`, `resetForTurn()` and `markTaskUsed()` to manage the task limit. The `markTaskUsed()` method also checks whether the staff member has already reached their task limit before incrementing the count. This helps ensure that each staff member performs at most one task per turn.

Overall: Your implementation is already sound. I would keep the current structure rather than introducing unnecessary complexity. For your SC2002 project, it provides a clear example of encapsulation that you can explain and demonstrate in your UML diagram and report.

Ask:
when i try to prepare an order, the prepareOrder() method keeps returning false even when the order status is PLACED or PREPARING

whats wrong with my logic & explain how to fix it?

below is my Chef.java code

```
package SC2002_Project.sc2002project;

public class Chef extends Staff {

    public Chef() {
        super();
    }

    // Prepare one unit of an order
    public boolean prepareOrder(Order order) {

        if (!canAct() || order == null) {
            return false;
        }

        OrderStatus status = order.getOrderStatus();

        // Only unfinished orders can be prepared
        if (status != OrderStatus.PLACED ||
            status != OrderStatus.PREPARING) {
            return false;
        }

        markTaskUsed();

        if (!order.prepareOneUnit()) {
            return false;
        }

        return true;
    }
}
```

ChatGPT:

The problem is in your **logical condition**, specifically your use of `||` (OR) instead of `&&` (AND).

Your current condition will **always evaluate to** `true`, regardless of whether the order is `PLACED` or `PREPARING`.

1. What's wrong with your current logic?

Your code:

```
None
if (status != OrderStatus.PLACED ||
    status != OrderStatus.PREPARING) {
    return false;
}
```

You're trying to say:

> If the order is neither PLACED nor PREPARING, don't allow preparation.

However, `||` means **OR**, so Java only needs one of those conditions to be true to enter the `if` statement.

Let's examine the possible statuses.

Order status	`status != PLACED`	`status != PREPARING`	OR (`||`)
`PLACED`	false	true	**true**
`PREPARING`	true	false	**true**
`READY`	true	true	**true**
`SERVED`	true	true	**true**

Notice that the overall expression is always `true`.

Therefore, your method always reaches `return false` before calling `order.prepareOneUnit()`.

2. How to fix it

Simply replace `||` with `&&`.

```
None
if (status != OrderStatus.PLACED &&
    status != OrderStatus.PREPARING) {
    return false;
}
```

Now you're saying:

> Reject the order only if its status is NOT PLACED **AND** NOT PREPARING.

Here's how it behaves:

Order status	`!= PLACED`	`!= PREPARING`	AND (`&&`)
`PLACED`	false	true	false (Allowed)
`PREPARING`	true	false	false (Allowed)
`READY`	true	true	true (Rejected)
`SERVED`	true	true	true (Rejected)

This correctly allows the Chef to prepare unfinished orders.

3. Another potential problem in your code

There's one more issue worth checking.

Your current implementation:

```
None
markTaskUsed();

if (!order.prepareOneUnit()) {
    return false;
}
```

You're marking the Chef's task as used **before confirming that preparation succeeded**.

Imagine `order.prepareOneUnit()` returns `false`.

Your method returns `false`, but the Chef has already consumed its task allowance.

That may violate your project's requirement that an invalid action should not consume the staff member's task.

Suggested improvement

Move `markTaskUsed()` after successful preparation.

```
None
public boolean prepareOrder(Order order) {

    if (!canAct() || order == null) {
        return false;
    }

    OrderStatus status = order.getOrderStatus();

    if (status != OrderStatus.PLACED &&
        status != OrderStatus.PREPARING) {
```

```
        return false;
    }

    if (!order.prepareOneUnit()) {
        return false;
    }

    markTaskUsed();
    return true;
}
```

This assumes `prepareOneUnit()` returns `true` when preparation successfully advances by one unit, and `false` otherwise.

4. What you've learned here

There are two important programming concepts demonstrated by this bug.

Logical operators:

- `||` (OR): At least one condition must be true.
- `&&` (AND): Both conditions must be true.
- When rejecting something that must not be *either* of two allowed values, use `&&` between the two `!=` comparisons.

State management:

Only consume a staff member's task allowance after a successful action, so invalid actions do not prevent the player from choosing again.

Final recommendation: Change `||` to `&&`, and move `markTaskUsed()` after `order.prepareOneUnit()` succeeds. These two small changes should fix your status-checking bug and improve your Chef's task management.

Ask: for seatCustomer() im not sure whether my conditions correctly prevent customers from being seated at already occupied tables or being seated again when theyre not in WAITING state

for takeOrder() sometimes i get ArrayIndexOutOfBoundsException when customer menu choice is invalid, but i already added a condition to check

whats wrong with my 2 methods?

below is my Waiter.java code

```
package SC2002_Project.sc2002project;

public class Waiter extends Staff {

    private int nextOrderID = 1;

    public Waiter() {
        super();
    }

    // Seat a customer at an available table
    public boolean seatCustomer(Customer cust, Table table) {

        if (!canAct() || cust == null || table == null) {
            return false;
        }

        if (table.isOccupied() &&
            cust.getStatus() != CustomerStatus.WAITING) {
            return false;
        }

        if (!table.occupy(cust)) {
            return false;
        }

        cust.changeStatus(CustomerStatus.SEATED);
        markTaskUsed();

        return true;
    }
```

```
// Take an order from a seated customer
public Order takeOrder(Customer cust, Table table, MenuItem[] menu) {

    if (!canAct() || cust == null ||
        table == null || menu == null) {
        return null;
    }

    if (cust.getStatus() != CustomerStatus.SEATED ||
        table.getOccupant() != cust ||
        table.getOrder() != null) {
        return null;
    }

    // Customer chooses their food
    int choice = cust.placeOrder();

    if (choice < 0 || choice > menu.length ||
        menu[choice] == null) {
        return null;
    }

    Order order = new Order(nextOrderID, table, menu[choice]);

    table.assignOrder(order);
    nextOrderID++;
    markTaskUsed();

    return order;
}

// Serve a completed order
public boolean serveOrder(Order order) {

    if (!canAct() || order == null) {
        return false;
    }

    if (order.getOrderStatus() != OrderStatus.READY) {
        return false;
```

```
        }

        Customer cust = order.getCustomer();

        if (cust == null ||
            cust.getStatus() != CustomerStatus.SEATED) {
            return false;
        }

        if (!order.serve()) {
            return false;
        }

        cust.changeStatus(CustomerStatus.EATING);
        markTaskUsed();

        return true;
    }
}
```

ChatGPT:

There are **two logical errors** in your `Waiter.java` code:

1. `seatCustomer()` — You used `&&` instead of `||`, allowing certain invalid seating actions.
2. `takeOrder()` — You used `>` instead of `>=`, allowing an invalid array index.

Both are common mistakes involving Boolean conditions and array boundaries.

1. Problem with `seatCustomer()`

Your current condition:

None

```
if (table.isOccupied() &&
    cust.getStatus() != CustomerStatus.WAITING) {
    return false;
}
```

What's wrong?

You are using `&&` (AND), which means **both conditions must be true** before the method rejects the seating attempt.

However, you should reject the action when **either condition is true**:

- The table is already occupied.
- The customer is not in the `WAITING` state.

Let's examine your current logic:

Table occupied?	Customer waiting?	Your condition rejects?	Correct behaviour
No	Yes	No	Allow
Yes	Yes	No	Reject
No	No	No	Reject
Yes	No	Yes	Reject

Your condition only rejects the last scenario.

This means a customer who is already seated could potentially be seated again at another empty table.

Although `table.occupy(cust)` might prevent seating at an occupied table, your Waiter's logic should still validate both conditions independently.

Corrected condition

```
None
if (table.isOccupied() ||
    cust.getStatus() != CustomerStatus.WAITING) {
    return false;
```

```
}
```

Using `||` means that if **either condition is invalid**, the method immediately returns `false`.

Now only customers in the `WAITING` state can be seated at unoccupied tables.

2. Problem with `takeOrder()`

Your current validation:

```
None
if (choice < 0 || choice > menu.length ||
    menu[choice] == null) {
    return null;
}
```

The problem is:

```
None
choice > menu.length
```

It should be:

```
None
choice >= menu.length
```

Why?

Java arrays use **zero-based indexing**.

Suppose your restaurant menu contains three items:

```
None
MenuItem[] menu = {
```

```
    salad,
    burger,
    pasta
};
```

The array looks like this:

Index 0

Salad

Index 1

Burger

Index 2

Pasta

First valid index: 0

Last valid index: 2

Index 3

Outside the array

Invalid

Even though `menu.length == 3`, the last valid index is `2`.

Your existing condition:

```
None
choice > menu.length
```

evaluates to `false` when `choice == 3`.

Java therefore proceeds to evaluate:

```
None
menu[choice] == null
```

Which becomes:

```
None
menu[3] == null
```

But `menu[3]` does not exist, causing an `ArrayIndexOutOfBoundsException`.

Corrected condition

```
None
if (choice < 0 || choice >= menu.length ||
    menu[choice] == null) {
    return null;
}
```

Now the valid range is:

$$0 \leq choice < menu.length$$

Because Java uses short-circuit OR evaluation, if `choice >= menu.length` is `true`, Java will not evaluate `menu[choice]`, preventing the exception.

3. Your corrected methods

You only need to change two operators. Everything else can remain the same for these two fixes.

Corrected `seatCustomer()`

```
None
public boolean seatCustomer(Customer cust, Table table) {

    if (!canAct() || cust == null || table == null) {
```

```
        return false;
    }

    if (table.isOccupied() ||
        cust.getStatus() != CustomerStatus.WAITING) {
        return false;
    }

    if (!table.occupy(cust)) {
        return false;
    }

    cust.changeStatus(CustomerStatus.SEATED);
    markTaskUsed();

    return true;
}
```

Corrected `takeOrder()`

```
None
public Order takeOrder(Customer cust, Table table, MenuItem[]
menu) {

    if (!canAct() || cust == null ||
        table == null || menu == null) {
        return null;
    }

    if (cust.getStatus() != CustomerStatus.SEATED ||
        table.getOccupant() != cust ||
        table.getOrder() != null) {
        return null;
```

```
    }

    // Customer chooses their food
    int choice = cust.placeOrder();

    if (choice < 0 || choice >= menu.length ||
        menu[choice] == null) {
        return null;
    }

    Order order = new Order(nextOrderID, table, menu[choice]);

    table.assignOrder(order);
    nextOrderID++;
    markTaskUsed();

    return order;
}
```

4. One additional thing to check

Your code assumes that `cust.placeOrder()` returns a **zero-based index**.

For example, if the customer chooses the first menu item, `placeOrder()` should return `0`, not `1`.

If your customer class uses menu choices numbered 1, 2 and 3, you must convert the selection to an array index before accessing `menu[choice]`.

I cannot verify this without seeing your `Customer.placeOrder()` implementation.

5. Test your corrected conditions

Try these test cases to confirm both methods work.

Testing checklist

0/8 checked

Seat waiting customer at empty table

Expected: true

Seat waiting customer at occupied table

Expected: false

Seat seated customer at empty table

Expected: false

Seat seated customer at occupied table

Expected: false

Take order with index 0

Expected: Order created

Take order with index 2 (3-item menu)

Expected: Order created

Take order with index 3 (3-item menu)

Expected: null, no exception

Take order with index -1

Expected: null, no exception

Make sure you reset the Waiter's task allowance between independent test cases and arrange the necessary customer/table state beforehand.

Main takeaway: For `seatCustomer()`, reject the action if *either* condition is invalid (`||`). For `takeOrder()`, remember that the maximum valid array index is `length - 1`, so use `>=` when checking the upper boundary.

These fixes also support your project's requirement that invalid actions should not consume the Waiter's task allowance.

Ask:
when a customer makes a payment, the no. of paid customers increase by 2 instead of 1 im not sure why this is happening because cashier should only handle 1 payment per turn

help me identify where the bug is & explain how to fix it

below is my Cashier.java and cashierAction() from RestaurantGame.java

```
public void cashierAction() {
   Cashier cash = (Cashier) staffList[0];
   cash.resetForTurn();

   while (cash.canAct()) {
      if (cash.handlePayment(tableList, custList, currStats)) {
         currStats.incrementPaidCustomers();
      }
   }
}
```

```
package SC2002_Project.sc2002project.Staff;

import SC2002_Project.sc2002project.Customer.*;
import SC2002_Project.sc2002project.Objects.*;
import SC2002_Project.sc2002project.*;

public class Cashier extends Staff {

   public Cashier() {
      super();
   }

   public boolean handlePayment(Table[] tableList,
         Customer[] custList, GameStats stats) {

      // Find oldest ready to pay customer
      for (int i = 0; i < 10; i++) {

         Customer cust = custList[i];
```

```
        if (cust != null &&
            cust.getStatus() == CustomerStatus.READY_TO_PAY) {

            Table selectedTable = tableList[cust.getTableID()];
            Order order = selectedTable.getOrder();

            int bill = order.getItem().getPrice();

            System.out.println("\nReceived $" + bill +
                " from Customer " + cust.getCustomerID() + "!");

            System.out.println("Table " +
                selectedTable.getTableID() + " is now freed up!");

            // Update stats
            cust.changeStatus(CustomerStatus.PAID);
            stats.increaseRevenue(bill);
            stats.incrementPaidCustomers();
            stats.updateTSatisfaction(cust.getSatisfaction());

            order.markPaid();
            selectedTable.release();
            markTaskUsed();

            return true;
        }
    }

    // No payment made
    markTaskUsed();
    return false;
  }
}
```

ChatGPT:

I found the bug. **You're incrementing the number of paid customers twice for every successful payment.**

The issue is not with your `Cashier` inheritance or the one-task-per-turn mechanism. It's because both `Cashier.java` and `RestaurantGame.java` are updating the same statistic.

1. Where is the bug?

In Cashier.java

Inside `handlePayment()`, you have:

```
None
// Update stats
cust.changeStatus(CustomerStatus.PAID);
stats.increaseRevenue(bill);
stats.incrementPaidCustomers(); // First increment
stats.updateTSatisfaction(cust.getSatisfaction());
```

This already increases the number of paid customers by 1.

In RestaurantGame.java

Inside `cashierAction()`, you have:

```
None
while (cash.canAct()) {
    if (cash.handlePayment(tableList, custList, currStats)) {
        currStats.incrementPaidCustomers(); // Second
increment
    }
}
```

When `handlePayment()` returns `true`, your controller increments the paid customer count **again**.

What happens during one payment?

1. Cashier finds a customer in `READY_TO_PAY` state.
2. `handlePayment()` processes the payment and increments the count from 0 to 1.
3. `handlePayment()` returns `true`.
4. `cashierAction()` sees `true` and increments the count from 1 to 2.

Actual payments

1

Recorded paid customers

2

Your Cashier is only handling one payment, but the statistic is being updated twice.

2. How to fix it

Remove `currStats.incrementPaidCustomers()` from `cashierAction()`.

Since `Cashier.handlePayment()` already updates the payment statistics, the controller doesn't need to do it again.

Corrected cashierAction() — RestaurantGame.java

```
None
public void cashierAction() {
    Cashier cash = (Cashier) staffList[0];
    cash.resetForTurn();

    while (cash.canAct()) {
        cash.handlePayment(tableList, custList, currStats);
    }
}
```

That's the only change required to fix the double-counting bug.

You can leave the following code in `Cashier.java` unchanged:

```
None
// Update stats
```

```
cust.changeStatus(CustomerStatus.PAID);
stats.increaseRevenue(bill);
stats.incrementPaidCustomers();
stats.updateTSatisfaction(cust.getSatisfaction());
```

3. One more improvement I recommend

Your current `cashierAction()` uses a `while` loop:

```
None
while (cash.canAct()) {
    cash.handlePayment(tableList, custList, currStats);
}
```

Although this works because `markTaskUsed()` is called inside `handlePayment()`, **a `while` loop is unnecessary when the Cashier can only perform one payment per turn.**

You can simplify the method to:

```
None
public void cashierAction() {
    Cashier cash = (Cashier) staffList[0];
    cash.resetForTurn();

    cash.handlePayment(tableList, custList, currStats);
}
```

This is what I would use for your project.

Why?

- The Cashier automatically processes at most one payment per turn.
- You don't need to repeatedly check whether the Cashier can act.
- The method is shorter and easier to understand.

- You avoid accidentally introducing an infinite loop if `handlePayment()` is modified later without consuming the task allowance.

Your `Cashier.java` already calls `markTaskUsed()` whether it finds an eligible customer or not, so the loop terminates in the current implementation.

One minor point: your `handlePayment()` loops through the first 10 customer-array entries. This is safe only if `custList` always contains at least 10 elements. Using `i < custList.length` would be more robust.

4. What OOP principle does this relate to?

This bug is also a useful example of **Single Responsibility Principle (SRP)** and clear ownership of state updates.

In your current design, both `RestaurantGame` and `Cashier` are responsible for incrementing the same statistic.

A cleaner division is:

Class	Responsibility
`RestaurantGame`	Controls when the Cashier performs its action.
`Cashier`	Processes eligible payments and updates the corresponding game statistics.
`GameStats`	Stores and manages revenue, paid customer count and satisfaction statistics.

This avoids having multiple classes independently updating the same information for one event.

5. Final corrected code

For your current implementation, I recommend keeping `Cashier.java` largely unchanged and replacing `cashierAction()` with:

```
None
public void cashierAction() {
    Cashier cash = (Cashier) staffList[0];
    cash.resetForTurn();

    cash.handlePayment(tableList, custList, currStats);
}
```

After this fix:

Successful payments	Before fix	After fix
1	2	1
2	4	2
3	6	3
4	8	4

This matters for your SC2002 project because the game's victory condition requires **at least 4 paid customers**. Your original bug could incorrectly award a victory after only 2 actual payments, assuming the other victory conditions were satisfied.

The corrected implementation ensures that every successful payment contributes exactly one to the paid customer count.

Ask: im working on Host and ComboMeal classes for stage 3 currently but im having some issues

for host, the host doesnt seem to be seating customers correctly even when theres an empty table & waiting customer, sometimes the customer status seems incorrect also, and choosing the wait option causes the game to get stuck

for combomeal, the combo prices & preparation units dun seem right, i think there is some issue with how the menu item returning

help me check the bugs for both classes & explain to me whats wrong

below is my Host.java and ComboMeal.java

```
package SC2002_Project.sc2002project.Staff;

import SC2002_Project.sc2002project.Customer.*;
import SC2002_Project.sc2002project.Objects.*;

public class Host extends Staff {

    public Host() {
        super();
    }

    // Seat one customer at an empty table
    public boolean seatCustomer(Customer cust, Table table) {
        if (!canAct()) {
            System.out.println("Host has already acted this turn.");
            return false;
        }

        if (cust == null && table == null) {
            System.out.println("Invalid customer or table.");
            return false;
        }

        if (!table.isOccupied()) {
            System.out.println("This table is occupied.");
            return false;
        }

        if (cust.getStatus() == CustomerStatus.WAITING) {
            System.out.println("This customer is not waiting.");
            return false;
        }
```

```
        table.occupy(cust);
        cust.changeStatus(CustomerStatus.WAITING);
        markTaskUsed();

        return true;
    }

    // Do nothing for this turn
    public void waiting() {

    }
}


package SC2002_Project.sc2002project.Objects;

public class ComboMeal extends MenuItem {

    private MenuItem firstItem;
    private MenuItem secondItem;

    public ComboMeal(MenuItem firstItem, MenuItem secondItem) {
        super(firstItem.getName() + " Combo",
            (int) Math.round((firstItem.getPrice() + secondItem.getPrice()) * 0.8),
            Math.max(firstItem.getPreparationUnits(), secondItem.getPreparationUnits()));

        this.firstItem = firstItem;
        this.secondItem = secondItem;
    }

    public MenuItem getFirstItem() {
        return secondItem;
    }

    public MenuItem getSecondItem() {
        return firstItem;
    }
}
```

ChatGPT:

I found **7 bugs across your** `Host.java` **and** `ComboMeal.java`. Most are small mistakes involving logical operators, reversed conditions and incorrect return values.

I'll explain each bug, why it causes your observed behaviour, and exactly what to change. I'll keep the corrected code beginner-friendly and consistent with your existing SC2002 project.

1. Host.java — 4 main bugs

Bug 1: Incorrect null checking

Your current code:

```
None
if (cust == null && table == null) {
    System.out.println("Invalid customer or table.");
    return false;
}
```

Problem: You used `&&` instead of `||`.

This condition only rejects the action when both `cust` and `table` are null.

If just one is null, your program continues and may throw a `NullPointerException`.

Fix:

```
None
if (cust == null || table == null) {
    System.out.println("Invalid customer or table.");
    return false;
}
```

Now the action is rejected if either object is null.

Bug 2: Reversed table occupancy condition

Your current code:

```
None
if (!table.isOccupied()) {
    System.out.println("This table is occupied.");
    return false;
}
```

Problem: The `!` operator reverses the Boolean value.

- `table.isOccupied()` returns `true` when occupied.
- `!table.isOccupied()` returns `true` when empty.

Your code is therefore rejecting empty tables instead of occupied ones.

Fix:

```
None
if (table.isOccupied()) {
    System.out.println("This table is occupied.");
    return false;
}
```

Bug 3: Reversed customer status condition

Your current code:

```
None
if (cust.getStatus() == CustomerStatus.WAITING) {
    System.out.println("This customer is not waiting.");
    return false;
}
```

Problem: You're rejecting customers who are actually waiting.

You want to reject customers who are *not* in the `WAITING` state.

Fix:

```
None
if (cust.getStatus() != CustomerStatus.WAITING) {
    System.out.println("This customer is not waiting.");
    return false;
}
```

With Bugs 2 and 3 fixed, seating is allowed only when both requirements are satisfied.

Table	Customer	Result
Empty	WAITING	Allow
Occupied	WAITING	Reject
Empty	SEATED	Reject
Occupied	SEATED	Reject

Bug 4: Incorrect status after seating

Your current code:

```
None
table.occupy(cust);
cust.changeStatus(CustomerStatus.WAITING);
markTaskUsed();
```

There are actually two issues here.

First, you change the customer's status to `WAITING` after seating them.

But once a customer is seated, their status should be `SEATED`.

Second, you're not checking whether `table.occupy(cust)` succeeded.

Since your earlier `Waiter.java` implementation treats `occupy()` as a Boolean method, you should check its return value before updating the customer.

Fix:

```
None
if (!table.occupy(cust)) {
    return false;
}

cust.changeStatus(CustomerStatus.SEATED);
markTaskUsed();
```

This ensures that the customer's status only changes after successful seating, and an unsuccessful seating attempt doesn't consume the Host's task.

2. Host.java — Why does choosing Wait cause the game to get stuck?

Your current method:

```
None
public void waiting() {

}
```

The method is completely empty.

This can cause an infinite loop depending on how you implemented the Host action in `RestaurantGame.java`.

For example, suppose your controller uses:

```
None
while (host.canAct()) {
```

```
    // Ask player for Host action
}
```

Here's what happens when the player selects Wait:

1. `host.canAct()` returns `true` because `tasksUsed = 0`.
2. Player selects Wait, calling `host.waiting()`.
3. The empty method does nothing, so `tasksUsed` remains 0.
4. `host.canAct()` still returns `true`, so the loop repeats.

The game may never exit the Host action loop.

Fix:

```
None
public void waiting() {
    markTaskUsed();
}
```

Although the Host isn't physically performing a task, this records that the Host's action for the turn has been used.

That way, `canAct()` returns `false` and the controller can proceed.

This is suitable if your game treats Wait as choosing to use up the current turn's action. Alternatively, the controller could explicitly exit its loop when Wait is selected.

3. Complete corrected Host.java

```
None

package SC2002_Project.sc2002project.Staff;

import SC2002_Project.sc2002project.Customer.*;
import SC2002_Project.sc2002project.Objects.*;
```

```
public class Host extends Staff {

    public Host() {
        super();
    }

    // Seat one customer at an empty table
    public boolean seatCustomer(Customer cust, Table table) {

        if (!canAct()) {
            System.out.println("Host has already acted this
turn.");
            return false;
        }

        if (cust == null || table == null) {
            System.out.println("Invalid customer or table.");
            return false;
        }

        if (table.isOccupied()) {
            System.out.println("This table is occupied.");
            return false;
        }

        if (cust.getStatus() != CustomerStatus.WAITING) {
            System.out.println("This customer is not
waiting.");
            return false;
        }

        if (!table.occupy(cust)) {
            return false;
```

```
        }

        cust.changeStatus(CustomerStatus.SEATED);
        markTaskUsed();

        return true;
    }

    // Do nothing for this turn
    public void waiting() {
        markTaskUsed();
    }
}
```

This implementation is also consistent with the task-management logic you previously implemented in `Staff.java`.

4. ComboMeal.java — 3 bugs

Your assignment specifies that a ComboMeal must:

- Contain two existing `MenuItem` objects.
- Cost **90% of their combined price**, rounded to the nearest cent.
- Require preparation units equal to the **sum** of its components.

Your current implementation violates the last two requirements.

Bug 5: Wrong discount percentage

Your code:

```
None
(int) Math.round(
    (firstItem.getPrice() + secondItem.getPrice()) * 0.8
)
```

Problem: You're multiplying by `0.8`, which means a 20% discount.

However, your Stage 3 assignment requires a 10% discount.

Fix:

```
None
(int) Math.round(
    (firstItem.getPrice() + secondItem.getPrice()) * 0.9
)
```

For example, combining Salad and Pasta:

Salad

$9.00

1 preparation unit

Pasta

$18.00

2 preparation units

COMBO RESULT

$24.30

$27.00 × 0.9

3 units

1 + 2

Important: This calculation assumes `getPrice()` returns integer cents, as required by your assignment. For instance, Salad should be stored as `900`, not `9`.

Bug 6: Incorrect preparation units

Your current code:

```
None
Math.max(firstItem.getPreparationUnits(),
         secondItem.getPreparationUnits())
```

Problem: `Math.max()` returns the larger value, not the sum.

For example:

```
None
Math.max(1, 2); // Returns 2
```

But combining Salad (1 unit) and Pasta (2 units) should require 3 preparation units.

Fix:

```
None
firstItem.getPreparationUnits() +
secondItem.getPreparationUnits()
```

This correctly calculates the total preparation units needed for the combo.

Bug 7: Your getters return the wrong items

Your current code:

```
None
public MenuItem getFirstItem() {
    return secondItem;
}

public MenuItem getSecondItem() {
    return firstItem;
}
```

Problem: The return values are swapped.

Calling `getFirstItem()` returns the second item, and vice versa.

Fix:

```
None
public MenuItem getFirstItem() {
    return firstItem;
}

public MenuItem getSecondItem() {
    return secondItem;
}
```

This mistake doesn't directly change the price calculation in your constructor, but can cause incorrect results in other parts of your program that use these getters.

5. Complete corrected ComboMeal.java

```
None

package SC2002_Project.sc2002project.Objects;

public class ComboMeal extends MenuItem {

    private MenuItem firstItem;
    private MenuItem secondItem;

    public ComboMeal(MenuItem firstItem, MenuItem secondItem)
{
        super(firstItem.getName() + " + " +
              secondItem.getName() + " Combo",
              (int) Math.round(
                  (firstItem.getPrice() +
secondItem.getPrice()) * 0.9
```

```
                ),
                firstItem.getPreparationUnits() +
                secondItem.getPreparationUnits());

        this.firstItem = firstItem;
        this.secondItem = secondItem;
    }

    public MenuItem getFirstItem() {
        return firstItem;
    }

    public MenuItem getSecondItem() {
        return secondItem;
    }
}
```

I also made a small improvement to the combo's name:

```
None
firstItem.getName() + " + " +
secondItem.getName() + " Combo"
```

Instead of just using `firstItem.getName() + " Combo"`, this displays both components.

For example, the customer sees `Salad + Pasta Combo`, making the contents clear.

One additional improvement would be to validate that neither component is null before building the combo. The code above assumes both menu items are valid, which is reasonable if your game creates combos only from its predefined menu.

6. Quick testing checklist

Test your fixes

0/8 passed

Host seats a WAITING customer at an empty table

Expected: true; customer becomes SEATED

Host tries an occupied table

Expected: false; no task consumed

Host tries to seat an already SEATED customer

Expected: false; no task consumed

Host selects Wait

Expected: Host action ends and game proceeds

Host tries another action after seating

Expected: false; one-task limit enforced

Combo: Salad + Pasta

Expected: $24.30; 3 preparation units

Combo: Burger + Pasta

Expected: $29.70; 3 preparation units

getFirstItem() and getSecondItem()

Expected: Return the matching components

7. Summary of all bugs

Location	Your code	Correction
Host null check	`cust == null && table == null`	Use `||`

Host table check	`!table.isOccupied()`	Remove `!`
Host customer check	`status == WAITING`	Use `!= WAITING`
Host seating	Set status to `WAITING`	Check `occupy()` result; set `SEATED`
Host waiting	Empty method	Call `markTaskUsed()`
Combo pricing	Multiply by `0.8`	Multiply by `0.9`
Combo preparation	`Math.max(a, b)`	Use `a + b`
Combo getters	Swapped items	Return matching items

There are eight distinct corrections in the table, with the seating checks and status update grouped under the same method-level problem.

My recommendation: Apply these corrections first, then run the test cases. The two classes are otherwise reasonably structured for a beginner Java OOP project.

One thing I would still want to verify is your `RestaurantGame.java` Host action logic. The empty `waiting()` method is a likely cause of the game getting stuck, but whether it fully fixes the problem depends on how your controller processes the player's Wait choice.